Neural Networks on FPGAs

Final Project — Student Instructions

Submission Deadline: 10 April 2025

1. Project Overview

Congratulations on reaching the final project! This is your chance to put everything you have learned together into something real — a neural network that actually runs on hardware.

Do not be intimidated. This document walks you through every step in simple, clear language. You will implement a small neural network in Verilog, generate its weights using Python, synthesise the design in Vivado, and finally run it on your FPGA board. Take it one day at a time.

What you will build

A 2-layer neural network implemented in Verilog that classifies simple inputs.
Weights and biases trained in Python (TensorFlow) and loaded onto the FPGA.
A working simulation that proves your design is correct before touching any hardware.
A short report describing what you did and what you observed.

2. How to Submit

Submission Deadline & Location

Submit everything on GitHub by 11:59 PM on 10 April 2025.
Create your folder at this exact path inside the repository:
ends_term/project/yourname_rollno/
Example: ends_term/project/mananjindal_230625/
Use lowercase. Replace 'yourname' with your first name and 'rollno' with your roll number.
Do NOT submit a zip file. Commit and push your files directly to GitHub.

2.1 Your Folder Structure

Create the following folders inside your submission folder. You do not need to fill them all on Day 1 — just create them and add files as you go.

```
ends_term/project/mananjindal_230625/
    src/          <- Your Verilog files go here
    sim/          <- Your testbench files go here
    weights/      <- The .mem files generated by Python
    python/       <- Your Python training script
    vivado/       <- Your Vivado project files
    docs/         <- Your report (PDF)
    README.md     <- A short description of your project
```

> *Tip: Make your first GitHub commit today with just the empty folders and a README. Committing regularly (every day) is better than one big commit on the last night.*

3. Week Plan

Here is a suggested day-by-day breakdown. You submit everything together on 10 April, but sticking to this schedule will keep you from panicking at the end.

Phase	Task	Day
Day 1	Set up Vivado, create your GitHub folder, push your first commit with a README.	1
Day 2	Run the Python script to train a tiny model and export weights/biases as .mem files.	2
Day 3	Write the neuron module in Verilog and simulate it. Make sure it works before moving on.	3
Day 4	Build a single layer by connecting neurons together. Test with a simulation.	4
Day 5	Combine both layers into a top-level module. Run Synthesis in Vivado.	5
Day 6	Flash the bitstream to your board and verify a few test inputs give the right output.	6
Day 7	Write your report, tidy up your code and README, and push everything to GitHub.	7

4. Step-by-Step Instructions

Task 1: Setup (Day 1)

Before any coding, get your tools and repository ready.

1. Open Vivado. Create a new RTL project (you can use a dummy top module for now just to verify it opens). If Vivado is not installed, follow the installation guide shared by your mentor.
2. On GitHub, navigate to the project repository. Inside it, create your personal folder: ends_term/project/yourname_rollno/. Add the sub-folders listed in Section 2.1.
3. Write a README.md. It only needs three lines right now: your name, your roll number, and one sentence describing what this project is. You will fill in the rest at the end.
4. Commit and push. That is it for Day 1.

Task 2: Python — Train the Model and Export Weights (Day 2)

You do not need to be a machine learning expert for this step. You will train a very small model on a simple dataset and save the numbers it learns (weights and biases) into files that Verilog can read.

What model to train

Use the Iris dataset (built into scikit-learn and TensorFlow datasets). Train a 2-layer network with this exact structure:

- Input: 4 features
- Hidden layer: 8 neurons, ReLU activation
- Output layer: 3 neurons, Softmax (3 flower classes)

Quantising the weights (converting floats to integers)

FPGAs work with integers, not decimals. After training, convert each weight and bias from a float to a 16-bit integer using this simple formula: multiply by 256 and round. This is called Q8 fixed-point format and is easier to work with than the Q4.12 format used in advanced designs.

```
# Example: convert a single weight
quantised = int(round(float_weight * 256))
quantised = max(-32768, min(32767, quantised))  # keep it in 16-bit range
```

Saving the .mem files

Save weights, biases, and 10 test inputs into text files in hex format, one number per line. Name them:

- weights/weights.mem
- weights/biases.mem
- weights/test_data.mem (10 input vectors + expected output label each)

Save your script as python/train_and_export.py. Running it should regenerate all three files from scratch.

> *Stuck on Python? Your mentor can provide a starter script with the model already defined.*
> *You just need to fill in the export section that saves the .mem files.*

Task 3: Verilog — The Neuron Module (Day 3)

A neuron does one thing: multiply each input by its weight, add them all up, add a bias, then pass the result through an activation function (ReLU). This is called a Multiply-Accumulate operation, or MAC.

You only need to write ONE Verilog module for this task: the neuron itself. We keep it simple — no pipeline, no FIFO, just a straightforward sequential design.

Module interface

Create src/neuron.v with the following ports:

```
module neuron (
    input  wire        clk,
    input  wire        rst_n,      // active-low reset
    input  wire        start,      // pulse high for 1 cycle to begin
    input  wire [15:0] data_in,    // one input value at a time (Q8 format)
    input  wire [15:0] weight_in,  // matching weight for that input
    input  wire [15:0] bias,       // bias value
    input  wire        last,       // pulse high on the final input
```

```
    output reg  [15:0] out,        // result after ReLU
    output reg         valid       // high for 1 cycle when output is ready
);
```

What the module should do

- On each clock cycle, multiply data_in by weight_in and add the result to an internal accumulator register.
- When last goes high, add the bias to the accumulator and apply ReLU: if the result is negative, output 0; otherwise output the result (truncated to 16 bits).
- Assert valid high for one cycle when the output is ready.

ReLU in Verilog

```
// Apply ReLU: output 0 if negative, else output the value
assign relu_out = (sum[31]) ? 16'd0 : sum[23:8]; // use bits [23:8] for Q8 truncation
```

Testbench

Write sim/tb_neuron.v. Drive the neuron with 4 known inputs and weights (you can calculate the expected output by hand or with Python). Check that valid goes high and that out matches the expected value. Use $display to print results and $finish to end the simulation.

> *Do NOT move to Task 4 until your simulation passes.*
> *A working neuron testbench is the most important checkpoint in this project.*

Task 4: Verilog — One Layer (Day 4)

A layer is just multiple neurons. For this project, you will implement one hidden layer with 8 neurons, all processing the same input in parallel (one neuron per output).

How to build it

Create src/layer.v. Inside it, instantiate 8 copies of your neuron module (one for each output). Each neuron gets the same data_in and start signals, but a different set of weights from the .mem file.

Load the weights using $readmemh. Here is how to declare a weight memory and load it:

```
reg [15:0] weight_mem [0:31]; // 8 neurons x 4 inputs = 32 weights
initial $readmemh("weights/weights.mem", weight_mem);
```

Then connect each neuron's weight_in to the right entry in weight_mem based on the current input index.

Layer testbench

Write sim/tb_layer.v. Feed one complete 4-value input vector into the layer and check that all 8 output values become valid and look reasonable (positive numbers for active neurons, 0 for inactive ones after ReLU).

Task 5: Top-Level Integration and Synthesis (Day 5)

5a. Top-level module

Create src/nn_top.v that connects your hidden layer (8 outputs) to a simpler output layer (3 neurons — one per class). The output neuron with the highest value is the predicted class.

Add two control signals: a start input that kicks off inference, and a done output that goes high when the result is ready.

5b. Running Synthesis in Vivado

5. Create a new Vivado project. Select your target board (e.g. Basys3 or Nexys A7).
6. Add all .v files from src/ to the project.
7. Add a constraints file (.xdc). You only need one line in it: the clock constraint.

```
create_clock -period 10.000 [get_ports clk]  ;# 100 MHz clock
```

8. Click Run Synthesis. Wait for it to complete (this takes a few minutes).
9. If you see red errors, fix them. Yellow warnings about unused signals are usually fine.
10. Click Run Implementation. When done, click Generate Bitstream.
11. Save the Vivado project files into your vivado/ folder and commit.

> *Common beginner synthesis errors:*
> *- 'Latch inferred' -> make sure every output is assigned in all branches of an if/else*
> *- 'Width mismatch' -> check that you are not connecting a 16-bit wire to an 8-bit port*
> *These are easy to fix once you read the error message carefully.*

Task 6: Hardware and Basic Verification (Day 6)

6a. Programming the board

12. Open Vivado Hardware Manager (Flow Navigator → Open Hardware Manager).
13. Connect your FPGA board via the USB cable and click Auto Connect.
14. Click Program Device and select your .bit file.
15. The board should power up. If you have an LED connected to your done signal, it should blink when inference finishes.

6b. Verify your results

Pick 5 test inputs from your test_data.mem file. Using switches or a UART interface (whichever your board setup uses), feed each input to the FPGA and note the output. Compare it to the expected label. Record your observations in a simple table like this:

Test #	Input Values	Expected Class	FPGA Output
1	[_, _, _, _]	Class _	Class _
2	[_, _, _, _]	Class _	Class _
3	[_, _, _, _]	Class _	Class _
4	[_, _, _, _]	Class _	Class _
5	[_, _, _, _]	Class _	Class _

6c. Two metrics to record (that is all!)

You only need to report two numbers. Find them in Vivado after synthesis completes:

- LUT utilisation — open the Utilisation Report in Vivado and find the percentage of LUTs used. Write it down.
- Timing slack (WNS) — open the Timing Summary Report. Find 'Worst Negative Slack'. If it is a positive number, your design meets timing. Write it down.

That is all the metrics you need. Include these two numbers in your report and README.

Task 7: Report and Final Cleanup (Day 7)

7a. Written Report

Write a short report (2–3 pages is fine) saved as docs/report.pdf. Do not overthink it — just answer these questions in your own words:

- What does your design do? (1 short paragraph)
- Draw a simple block diagram showing the neuron, layer, and top-level module connected together.
- What results did you get? (paste your 5-test verification table and the 2 metrics from Task 6)
- What was the hardest part for you? What did you learn?

7b. Complete your README.md

Update your README.md to include:

- Your name and roll number
- Your FPGA board name
- A one-paragraph description of the project
- How to run the Python script (one command)
- How to open and program in Vivado (2–3 sentences)
- Your 2 metrics: LUT utilisation and timing slack

5. Final Checklist Before Submitting

Go through this list before your last push. If something is missing, add it.

File	Description	Notes
`README.md`	Name, roll no, board, description, metrics	*Required*
`src/neuron.v`	Single neuron module (MAC + ReLU)	*Required*
`src/layer.v`	One hidden layer (8 neurons instantiated)	*Required*
`src/nn_top.v`	Top-level connecting hidden + output layer	*Required*

`sim/tb_neuron.v`	Neuron testbench — must show correct output	*Required*
`sim/tb_layer.v`	Layer testbench	*Required*
`python/train_and_export.py`	Python script that generates .mem files	*Required*
`weights/*.mem`	weights.mem, biases.mem, test_data.mem	*Required*
`vivado/`	Vivado project files (.xpr)	*Required*
`docs/report.pdf`	2–3 page report with block diagram and results	*Required*

6. Grading Breakdown

Component	Marks	Notes
Python script + correct .mem files generated	15	Script must run without errors
Neuron module + passing testbench simulation	25	Simulation must show correct output
Layer module (8 neurons, weights loaded from .mem)	20	Must synthesise cleanly in Vivado
Top-level integration + bitstream generated	15	Bitstream must be present in vivado/
Hardware verification (5 test inputs checked)	10	Fill in the table from Task 6
Report (block diagram + results + reflection)	10	2–3 pages, honest and clear
README + code comments + commit history	5	Shows understanding of the work
Total	**100**	

7. Rules

- Write your own Verilog. Do not copy-paste full modules from the internet or from classmates.
- You can discuss ideas and debug together, but what you submit must be written by you.
- You can use AI tools (ChatGPT, etc.) to understand a concept or debug an error message, but you must understand and be able to explain every line you submit.
- Late submissions lose 10 marks per day.

8. Getting Help

Getting stuck is normal and expected. Here is how to get help quickly:

- Simulation not working? Post your tb_neuron.v code and the exact error in the group chat. Do not post screenshots of code — paste the text.
- Vivado synthesis error? Copy the full error message text and search it. Most errors have a clear fix.
- Hardware not responding? Double-check that you have correctly set the clock constraint and that the bitstream was generated after your last code change.
- Not sure about the .mem file format? Ask your mentor for a sample — seeing one correct example is worth more than reading the documentation.

How to reach your mentor

Open a GitHub Issue in the project repository for questions that might help the whole class.

For personal help, message your mentor directly and include a link to your GitHub folder.

Always describe: (1) what you tried, (2) what you expected, (3) what actually happened.

You have got this. One task at a time.